Alla scoperta di GIT

11 aprile 2021



1. Cos'è GIT
2. Architettura di GIT
3. GIT vs SVN
4. Installazione di GIT
5. Alcuni esempi
6. Workflow collaborativo
7. Fare i giocolieri con i commit
8. Utilizzo di GitHub, GitLab e Bitbucket
9. GIT e Jetbrains
10. Cosa non fa GIT
11. Collaborazione attraverso GitHub, GitLab e Bitbucket

Cos'è GIT

GIT è un software di controllo della versione centralizzato. Il controllo di versione (VCS - Version Control System) è un software che registra i cambiamenti di un file nel corso del tempo. Servono per gestire le modifiche del software nel tempo. Vengono registrate tutte le modifiche, si può tornare indietro e si può confrontare il codice.
Quando vengono fatte delle modifiche al progetto le modifiche non vengono direttamente caricare all'interno dell'archivio, ma si aspetta un commit e questo commit contiene una descrizione sintetica delle modifiche, oltre che alle informazioni sull'autore della modifica

Storia di GIT

Git è nato nel 2005, per gestire il codice sorgente del kernel di Linux; Scritto da Linus Torvalds, che voleva un sistema distribuito che potesse usare come BitKeeper, ma nessuno dei sistemi disponibili gratuitamente soddisfaceva i suoi bisogni, particolarmente il suo bisogno di velocità. BitKeeper non lo poteva più usare gratuitamente in quanto Andrew Tridgell aveva effettuato il reverse engineering dei protocolli.
È un sistema di version control distribuito, pensato per gestire grandi cartelle di codice sorgente ma con la possibilità di gestire qualsiasi tipo di file.

Il nome GIT

Cosa può significare GIT?

1. 3 lettere casuali mai usate dagli OS UNIX quindi utilizzabili.
2. nello slang americano significa idiota/stupido
3. global information tracker ma solo quando sei bravo (monitoraggio delle informazioni globali)
4. goddamn idiotic truckload (dannato carico di camion idiota) ma solo quando non funziona

Linus affermò che ha scelto GIT propio perché nello slang americano significa idiota/stupido e gli sembrava divertente.

L'utilità di un VCS

Come già detto serve per vedere nel tempo le modifiche del software, ma perché è importante?

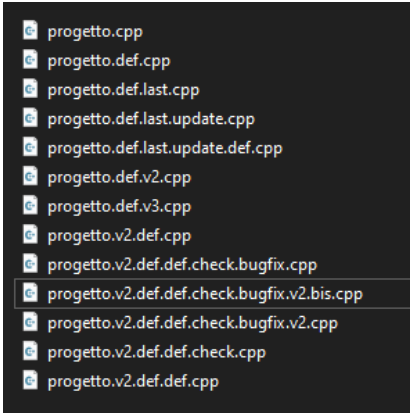
1. Ti permette di ripristinare file alle versione precedenti per rimuovere modifiche errate o non volute
2. Ti permette di vedere la cronologia delle modifiche ai file e quindi raccoglie le conoscenze del team sullo sviluppo, magari è stata modificata una funzione perché ha un bug, ma data la memoria corta viene rifatto il codice uguale a prima, dimenticandosi dell'altro bug.
3. Il controllo della versione protegge il codice sorgente sia dalla catastrofe sia dal degrado casuale dell'errore umano e delle conseguenze indesiderate.
4. Gli sviluppatori di software che lavorano in team scrivono continuamente nuovo codice sorgente e modificano il codice sorgente esistente: magari il codice non è perfettamente aggiornato o dopo una semplice modifica tutti gli sviluppatori non riescono più a compilare
5. Ti permette di effettuare operazioni "merge" tra i vari rami del progetto segnalando eventuali conflitti

L'utilità di un VCS

6. Il codice per un componente di progetto, app o software è in genere organizzato in una struttura di cartelle o "albero dei file", dove uno sviluppatore del team potrebbe lavorare su una nuova funzionalità mentre un altro sviluppatore corregge un bug non correlato modificando il codice, ogni sviluppatore può apportare le proprie modifiche in diverse parti dell'albero dei file.

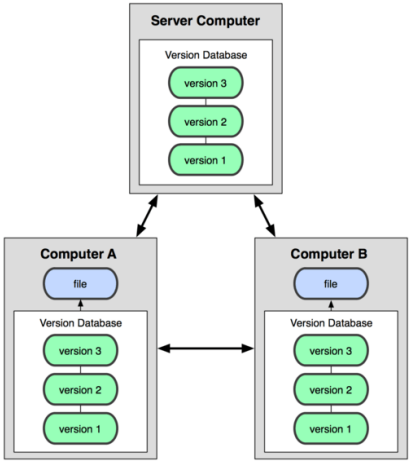
Il controllo della versione aiuta i team a risolvere questo tipo di problemi, aumentando le capacità di sviluppo contemporaneo e simultaneo, riducendo i conflitti e avendo uno storico delle modifiche. Sebbene sia possibile sviluppare software senza utilizzare alcun controllo di versione, ciò comporta un enorme rischio per il progetto che nessun team di professionisti dovrebbe accettare. Quindi la domanda non è se usare il controllo versione ma quale sistema di controllo versione usare.

L'utilità di un VCS



Ecco un motivo per cui è meglio un repository GIT.
P.S. ma qual'è l'ultimo file, quello definitivo?

DVCS



Nei Sistemi di Controllo di Versione Distribuiti (Distributed Version Control Systems) i client non solo controllano la versione più recente dei file, ma fanno una copia completa del repository.

Come scegliere un VCS

Cosa deve avere un VCS perché sia valido?

1. **Una cronologia completa delle modifiche a lungo termine di ogni file.** Deve poter memorizzare tutti i cambiamenti apportati da molte persone nel corso degli anni. Le modifiche includono la creazione e la cancellazione di file, nonché modifiche al loro contenuto. Questa cronologia dovrebbe includere anche l'autore, la data e le note scritte sullo scopo di ogni modifica. Avere la cronologia completa consente di tornare alle versioni precedenti per aiutare nell'analisi della causa principale dei bug ed è fondamentale quando è necessario risolvere i problemi nelle versioni precedenti del software. Se il software viene lavorato attivamente, quasi tutto può essere considerato una "versione precedente" del software.
2. **Ramificazione e fusione.** La creazione di rami permette di far lavorare i membri del team contemporaneamente senza infastidirsi a vicenda, ed ogni volta che i vari pezzi di codice sono stati completati si "fondono" insieme, e far si che chi fonde sia in grado di gestire solamente i veri conflitti, evitando di dover controllare tutti i file anche per minuscole differenza

Come scegliere un VCS

4. **Tracciabilità.** Essere in grado di tracciare ogni modifica apportata al software e essere in grado di annotare ogni modifica con un messaggio che descrive lo scopo e l'intento della modifica può aiutare l'analisi della causa principale del bug. Avere la cronologia annotata del codice a portata di mano quando si legge il codice, cercando di capire cosa sta facendo e perché è così progettato può consentire agli sviluppatori di apportare modifiche corrette e armoniose in accordo con il progetto a lungo termine previsto del sistema. Ciò può essere particolarmente importante per lavorare in modo efficace con il codice legacy ed è cruciale per consentire agli sviluppatori di stimare il lavoro futuro con precisione.

Nei Sistemi di Controllo di Versione Distribuiti (Distributed Version Control Systems) i client non solo controllano la versione più recente dei file, ma fanno una copia completa del repository. Il motivo è semplice: dato che i file pesano poco, usare qualche bit in più a contenere le versioni precedenti evita la connessione continua al server per scariare il progetto aggiornato, inoltre permette di lavorare anche offline e di avere più backup.

“Un bug non è mai solo un errore. Rappresenta qualcosa di più. Un errore nel modo di pensare. Perché, fuori dallo schermo, la vita è piena di bug e ti rende come sei. Il bug costringe il software ad adattarsi, per causa sua deve evolvere in qualcosa di nuovo. Deve aggirare l'ostacolo o superarlo. Qualsiasi cosa accada si trasforma. Diventa qualcosa di nuovo. La versione successiva. L'inevitabile upgrade. Mr. Robot - Elliot”

Best practice

4. **Fare spesso commit.** I commit sono facili e veloci. Dovrebbero essere eseguiti frequentemente per acquisire gli aggiornamenti di una base di codice. Ogni commit è un'istantanea su cui è possibile ripristinare la base di codice se necessario. I commit frequenti offrono molte opportunità di ripristinare o annullare il lavoro. Un gruppo di commit può essere combinato in un singolo commit usando un rebase per chiarire il registro di sviluppo.
5. **Programma sempre sull'ultima versione.** i VCS permettono aggiornamenti rapidi da più sviluppatori. È facile avere una copia locale non sincronizzata, rendendo più difficile il merge!
6. **Dettaglia bene le modifiche.** Ogni commit ha una voce di registro corrispondente. Al momento della creazione del commit, questa voce di registro viene popolata con un messaggio. È importante lasciare messaggi di registro di commit esplicativi descrittivi.

Best practice

5. **Usa i rami.** Il ramo è un potente meccanismo del VCS che consente agli sviluppatori di creare una linea di sviluppo separata. I rami dovrebbero essere usati frequentemente in quanto sono rapidi ed economici.
6. **Concorda un metodo unico per la gestione del flusso.** Si decide una linea guida per fare le fusioni, per la creazione di rami e tutto!

1 Cos'è GIT

2 Architettura di GIT

3 GIT vs SVN

4 Installazione di GIT

5 Alcuni esempi

6 Workflow collaborativo

7 Fare i giocolieri con i commit

8 Utilizzo di GitHub, GitLab e Bitbucket

9 GIT e Jetbrains

10 Cosa non fa GIT

11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Design di GIT

Il design di Git è stato basato su 5 punti:

- Velocità
- Semplicità di design e uso
- Supporto per sviluppo non lineare (migliaia di versioni parallele)
- Completamente distribuito
- Abilità di gestire grandi progetti

La differenza principale tra Git e altri VCS sta in come Git pensa e vede i propri dati.

- Molti VCS (come CVS e Subversion) vedono i dati che gestiscono come un set di file e di cambiamenti apportati a tali file nel tempo (delta-based version control)
- Git vede i dati come una serie di snapshot di un filesystem in miniatura.

Gli "oggetti" di Git

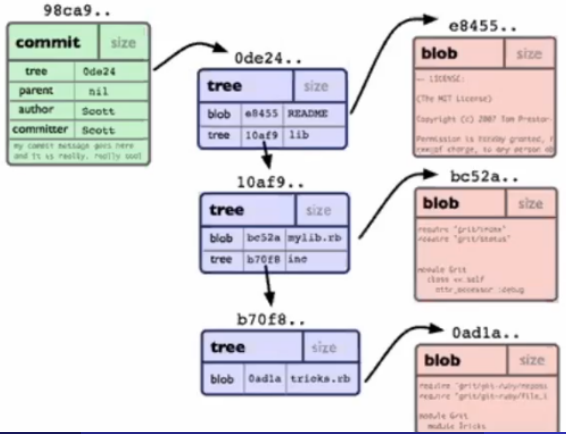
Git è basato principalmente su due strutture dati:

- Un indice mutabile (chiamato stage o cache) nel quale vengono raccolte tutte le informazioni sulla directory di lavoro
- Un object database immutabile che può contenere 4 tipi di oggetti (blob, tree, commit, tag)

Un **blob** (Binary Large Object) rappresenta il contenuto di un file. Ciascun blob all'interno dell'object database di Git è rappresentato dal proprio hash.
Un "**albero**" è l'equivalente di una directory; contiene una lista di nomi di file e la relativa referenza all'oggetto, sia esso un file, link simbolicoo una directory.

Gli "oggetti" di Git

Un **commit object** è un oggetto che collega vari tree object formando una storia; contiene il nome del tree object, un timestamp, un messaggio di log, il nome e l'email dell'autore e un puntamento ad un eventuale commit precedente



Gli "oggetti" di Git

Un "**tag**" contiene un puntatore ad un qualsiasi altro oggetto presente nel database e i suoi metadati



1 Cos'è GIT

2 Architettura di GIT

3 GIT vs SVN

4 Installazione di GIT

5 Alcuni esempi

6 Workflow collaborativo

7 Fare i giocolieri con i commit

8 Utilizzo di GitHub, GitLab e Bitbucket

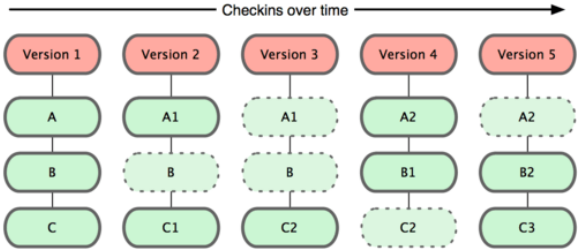
9 GIT e Jetbrains

10 Cosa non fa GIT

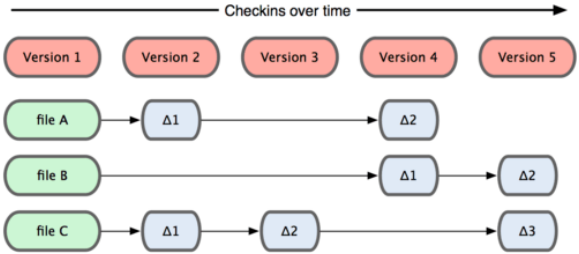
11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Differenze e Snapshot

Snapshot Ogni volta che si salva lo stato di un progetto vengono fatte delle istantanee di tutti i file e salvate nel sistema VCS



Differenze Ogni volta che si salva lo stato di un progetto vengono salvati nel sistema VCS le differenze rispetto al file precedente



Git e Svn

GIT è un DVCS creato da Linus Torvalds nel 2005 per semplificare le operazioni di gestione del Reposity del Kernel Linux.
Utilizza le snapshot per salvare le modifiche.

Sub**V**ersio**N** è un VCS creato per sostitute il CVN, ritenuto ormai obsoleto.
Utilizza le differenze per salvare le modifiche.

Git e Svn

GIT

- Distribuito
- Copie locali del reposity dove lavorare
- Si può limitare l'accesso a determinate direcotry
- Cronologia delle modifiche disponibili anche offline
- Connessione di rete necessaria solo alla sincronizzazione

SubVersioN

- Centrale
- Copia locale dell'ultima versione dove lavorare
- Non si può limitare l'accesso alle directory
- Cronologia delle modifiche non disponibile offline
- Connessione di rete sempre neccessaria

Git e Svn

GIT

- Git non permette di recuperare in maniera facile la cronologia delle modifiche dopo aver rinominato il file
- Supporto a progetti di centinaia di Gigabyte tramite il GVFS
- Git non adora i file binari
- Esistono fornitori di reposity Git gratuiti tra cui GitHub, GitLab e Bitbucket che hanno una elevata fama

SubVersioN

- Nonostante non sia un sistema distribuito è facile fare copie di backup
- Pochi comandi, semplici e intuitivi
- Impossibile fare una copia locale se non puoi accedere al server SVN
- I fornitori di reposity SVN gratuiti non sono famosi come quelli di Git. E non sono affidabili come quelli che forniscono Git

Git e Svn: i numeri

	Subversion	**Git**
Data Source	https://core.svn.wordpress.org/	https://github.com/wordpress/wordpress
Numero di Revisioni	42704	38070
Dimensione del reposity (Dati non aggiornati)	195,153,948 bytes	177,922,471 bytes
Software Version	Subversion 1.9.2, 64 bit	Git 2.6.3, 64 bit
Commento	Il reposity è il dump del reposity ufficiale di wordpress	Il reposity viene mantenuto da GitHub, alcune revisioni vengono ommesse

Git e Svn: dove non usarli

GIT

- L'utilizzo di GIT è sconsigliato in progetti di medie/grandi dimensioni in quanto le principali operazioni diventano molto più lente

SubVersioN

- L'utilizzo di SVN è sconsigliato nei progetti dove ci sono molte modifiche in parallelo sugli stessi file in quanto può avere solo una singola cronologia lineare e confonderlo è molto facile

GIT+SVN

- GIT+SVN è un approcio utilizzare in maniera "biridezionali" i vantaggi (e gli svantaggi) dei reposity GIT e SVN

GVFS

- Git Virtual File System (GVFS) consente a Git di operare su scala aziendale. È possibile utilizzare e gestire enormi repository Git.

Mercurial

- Mercurial è un'alternativa a GIT e SVN.
- Utilizza un sistema ibrido per il salvataggio degli stati:
- Utilizza un algoritmo di delta quando le differenze sono poche, altrimenti un'istantanea.

Git e Svn


git
SUBVERSION
Final 2
Final Final
FINAL FOR REAL THIS TIME
OK This is definitely final
I prefer the real version control
I said the *real* version control
Perfection

1 Cos'è GIT
2 Architettura di GIT
3 GIT vs SVN
4 Installazione di GIT
5 Alcuni esempi
6 Workflow collaborativo
7 Fare i giocolieri con i commit
8 Utilizzo di GitHub, GitLab e Bitbucket
9 GIT e Jetbrains
10 Cosa non fa GIT
11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Installazione di GIT su Linux

Per installare GIT su un sistema operativo tipo Ubuntu, Debian

```
$ sudo apt update
$ sudo apt install git
```

Dopo di che dal termianle verificare che l'installazione è andata a buon fine:

```
$ git --version
```

Impostiamo le nostre credenziali:

```
$ git config --global user.name "Emma Paris"
$ git config --global user.email "eparis@atlassian.com"
```

Ricordo di non usare credenziali che non conducano espressivamente alla persona che ha scritto il codice (furbissimo@gmail.com) in quanto queste credenziali restano salvate per sempre, come se fosse blockchain o la cronologia di Wikipedia, nonostante ciò è possibile rimuovere i commit e modificare la cronologia (con un po' di difficoltà e la rigenerazione di hash, anche con l'uso di BFG Repo-Cleaner)

Installazione di GIT su Windows

Scarica da qui l'ultima versione: `https://gitforwindows.org/`
Dopo di che aprire il terminale e verificare che l'installazione è andata a buon fine:

```
$ git --version
```

Impostiamo le nostre credenziali:

```
$ git config --global user.name "Emma Paris"
$ git config --global user.email "eparis@atlassian.com"
```

Ricordo di non usare credenziali che non conducano espressivamente alla persona che ha scritto il codice (furbissimo@gmail.com) in quanto queste credenziali restano salvate per sempre, come se fosse blockchain o la cronologia di Wikipedia, nonostante ciò è possibile rimuovere i commit e modificare la cronologia (con un po' di difficoltà e la rigenerazione di hash, anche con l'uso di BFG Repo-Cleaner)

Installazione di GIT su macOS

Installa XCode
Dopo di che aprire il terminale e verificare che l'installazione è andata a buon fine:

```
$ git --version
```

Impostiamo le nostre credenziali:

```
$ git config --global user.name "Emma Paris"
$ git config --global user.email "eparis@atlassian.com"
```

Ricordo di non usare credenziali che non conducano espressivamente alla persona che ha scritto il codice (furbissimo@gmail.com) in quanto queste credenziali restano salvate per sempre, come se fosse blockchain o la cronologia di Wikipedia

Salvataggio delle credenziali offline

Se usiamo la riga di comando per fare commit, ci chiederà ogni volta le credenziali del server. Per salvare queste credenziali nel PC:

```
$ git config credential.helper store
$ git push https://example.com/repo.git
Username: <type your username>
Password: <type your password>
```

Attenzione, spesso quando si digita la password su terminali unix-like il terminale non da nessun feed per la digitazione delle password.

Attenzione questo metodo salva le credenziali in un file di testo (plaintext) non cripato che è il `global .gitconfig file`, accessibile da tutti nel pc, su linux si trova in `/home/[username]/.gitconfig`. Se non si desidera questo comportamento è possibile:

- Impostare un timeout del file
  `git config credential.helper cache <timeout>`, dove il timeout è scritto in secondi ed è opzionale con valore di default a 900 secondi, cioè 15 minuti
- Usare una chiave SSH

Dopo aver creato la chiave SSH, per esempio su GitHub: si va sui Settings → SSH and GPG keys → New SSH Key, poi si salva la chiave sul proprio pc. Dopo di che salviamo la nostra chiave `id_rsa` nella directory `~/.ssh/`. Ed infine la aggiungiamo per l'autenticazione:

```
ssh-add -K ~/.ssh/id_rsa
```

Su Windows la cartella di default per salvare le credenziali SSH è:
`C:\Users\[username]\.ssh\`

Creazione di un nuovo repository

crea una nuova directory, entraci ed esegui

```
$ git init
```

per creare un nuovo repository git.
Per eseguire una copia di un repository locale eseguendo il comando

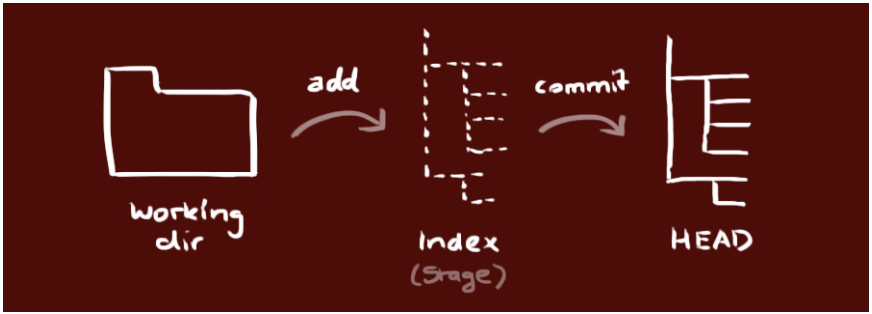
```
$ git clone /percorso/del/repository
```

Invece per scaricare un repository git dal web, il comando sarà:

```
$ git clone https://github.com/WordPress/WordPress.git
```

Ambiente di lavoro

la tua copia locale del repository è composta da tre "alberi" mantenuti da git. Il primo è la tua Directory di lavoro che contiene i files attuali. Il secondo è l'Index che fa da spazio di transito per i files e per finire l'HEAD che punta all'ultimo commit fatto.

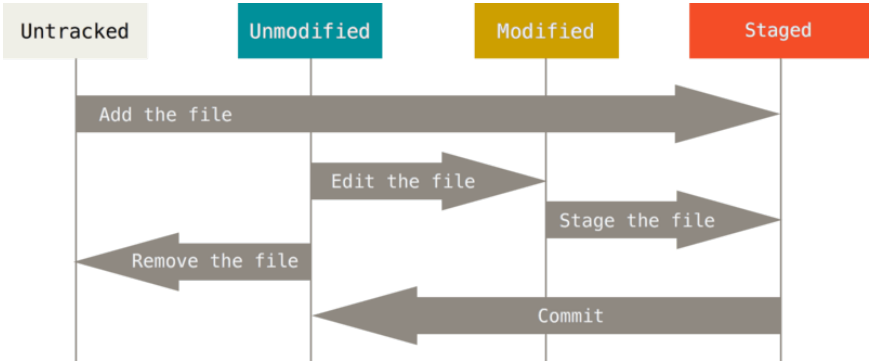


Stati

Analizziamo gli stati in cui si può trovare un file controllato da Git:

1. Modified: il file è stato modificato rispetto al blob presente sul database, ma non è stato ancora salvato su quest'ultimo.
2. Staged: la versione 'modified' del file è stato selezionata dall'utente per essere presente nella prossima commit.
3. Commmited: la versione 'staged' del file è stata correttamente salvata nel database.
4. Untracked: il file non è presente nel database e non è stato aggiunto ai file in stage

Stati


Untracked
Unmodified
Modified
Staged
Add the file
Edit the file
Stage the file
Remove the file
Commit

Workflow di base

Come per gli stati, il workflow di base di Git prevede tre step, ciascuno assiociato ad uno stato dei file della working directory:

1. L'utente effettua delle modifiche sui file controllati da Git.
2. L'utente seleziona e inserisce nell'area di "stage" le modifiche che vuole facciano parte del prossimo commit.
3. L'utente effettua una commit; vengono presi tutti i file nell'area di "stage" e salvati permanentemente nel database

Salviamo le modifiche

Puoi proporre modifiche (aggiungendole all'Index cioè la lista dei file in attesa di commit) usando

```
$ git add <nomedelfile>
$ git add *
$ git add -i
```

Il secondo passo è quello di creare un commit, cioè per validare queste modifiche fatte si usa

```
$ git commit -m "Messaggio per la commit"
```

Ora abbiamo salvato le modifiche e queste modifiche saranno salvate in locale, non ci resta che sincronizzarli con il server remoto

```
$ git push origin master
```

Aggiungere un server remoto

Se non hai copiato un repository esistente, e vuoi connettere il tuo repository ad un server remoto, c'e' bisogno che tu lo aggiunga con

```
$ git remote add origin <server>
```

Ora sarai in grado di inviare le tue modifiche al server remoto specificato

Scarichiamo le modifiche

per aggiornare il tuo repository locale alla commit più recente, esegui

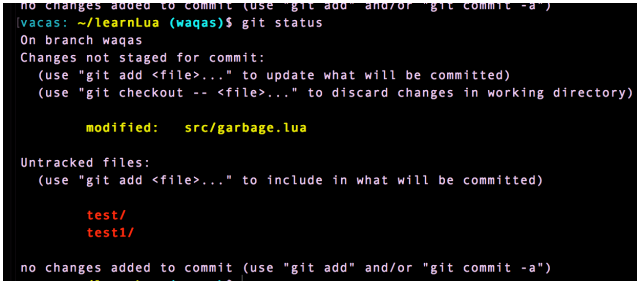
```
$ git pull
```

nella tua directory corrente per fare una fetch (recuperare) ed incorporare (merge), se possibile, le modifiche fatte sul server remoto.

Modifiche in corso

Per vedere lo stato delle modifiche in corso possiamo usare il comando

```
$ git status
```

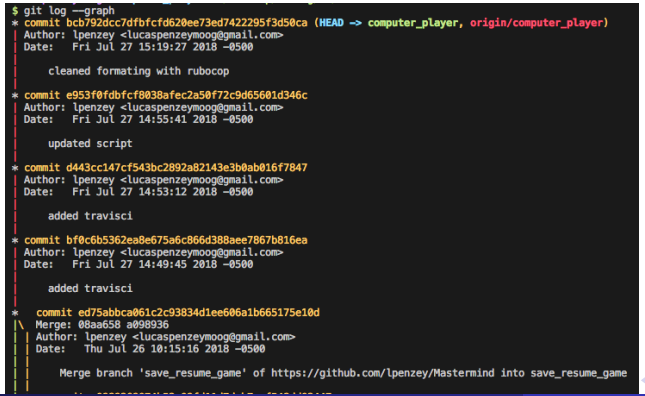


Lista delle modifiche

Per vedere la lista delle modifiche effettuate nel corso del tempo

```
$ git log
```

L'opzione `--graph` mostra anche l'albero cronologico dei commit, molto utile nel caso di un repo con molti branch.

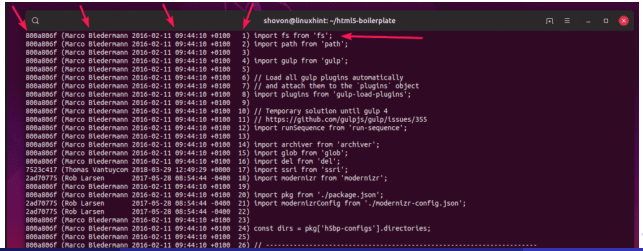


Chi ha fatto chi

Per sapere chi ha modificato, per l'ultima volta una riga di un file possiamo utilizzare il comando

```
$ git blame README.MD
```

git blame opera solamente su file singoli, è quindi richiesta una path di un file. L'opzione `-M` rileva le linee spostate o copiate all'interno dello stesso file. In questo modo viene riportato l'autore originale delle righe invece dell'ultimo autore che ha spostato o copiato le righe.
Il nome dal suono negativo deriva dal fatto che questo comando viene spesso usato per incolpare le persone che hanno rotto un programma, anche se il suo utilizzo è molto più ampio




the most efficient way
to get rid of all the bugs
was to get rid
of all the software,

Sembra molto complicato rispetto a caricare i file su Dropbox





git

Ctrl + Z
Ctrl + Z
Ctrl + Z
Ctrl + Z
Ctrl + Z
Ctrl + Z
Ctrl + Z

Ecco se usiamo servizi come Dropbox, Google Drive, Onedrive, Box, ... se troviamo un codice con tanti bug ci viene in mente di eliminarlo e non di correggerlo!

Inoltre l'utilizzo di GIT (o software similari) è una routine in qualsiasi azienda di programmazione, meglio impararlo prima che dopo!

E ricordo la storia delle modifiche è importante e più utile di fare Ctrl+Z!

IDE & .gitignore

Iniziamo con il fatto che nel repository .git non ci va per nessun motivo, ma per nessun motivo un file compilato, neanche file di configurazione locali degli IDE e qualsiasi altra cosa che è legata al PC e non al progetto software. Git mette a disposizione il file dot gitignore, che è un file dove vengono specificati che file ignorare. Una collezzione di file dot gitignore è presente su questo repository git: `https://github.com/github/gitignore`
In linea generale, consiglio di mettere i appendere i seguenti file gitignore in uno solo: Jetbrains, Visual Studio, Eclipse e quello del linguaggio di programmazione del repository (in questo caso C++)
Dopo aver modificato il file dot gitignore possiamo usare questo comando per "pulire" il repository

```
$ git rm -r --cached .
$ git add .
$ git commit -am "Remove ignored files"
```

1 Cos'è GIT

2 Architettura di GIT

3 GIT vs SVN

4 Installazione di GIT

5 Alcuni esempi

6 Workflow collaborativo

7 Fare i giocolieri con i commit

8 Utilizzo di GitHub, GitLab e Bitbucket

9 GIT e Jetbrains

10 Cosa non fa GIT

11 Collaborazione attraverso GitHub, GitLab e Bitbucket

In case of fire



1. git commit
2. git push
3. leave building

Scarichiamo un repository vuoto

```
$ git clone https://gitlab.com/test/test.git
$ cd test
$ touch README.md
$ git add README.md
$ git commit -m "add README"
$ git push -u origin master
```

Caricare una cartella esistenze

```
$ cd existing_folder
$ git init
$ git remote add origin https://gitlab.com/test/test.git
$ git add .
$ git commit -m "Initial commit"
$ git push -u origin master
```

Caricare un repo git esistente

```
$ cd existing_repo
$ git remote rename origin old-origin
$ git remote add origin https://gitlab.com/test/test.git
$ git push -u origin --all
$ git push -u origin --tags
```

1 Cos'è GIT

2 Architettura di GIT

3 GIT vs SVN

4 Installazione di GIT

5 Alcuni esempi

6 Workflow collaborativo

7 Fare i giocolieri con i commit

8 Utilizzo di GitHub, GitLab e Bitbucket

9 GIT e Jetbrains

10 Cosa non fa GIT

11 Collaborazione attraverso GitHub, GitLab e Bitbucket

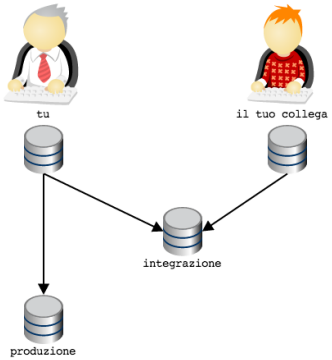
Workflow

Quando si sviluppa in più persone si potrebbe pensare di tenere tutti i file su un server NAS e di modificarli lì, nessuna scelta è peggiore di questa: in quanto se uno sta facendo una modifica l'altro non può compilare, inoltre il NAS non è detto che gestisca bene modifiche multiple e tutto, ci sono altre soluzioni intermedie, sempre inadeugate. La soluzione è l'uso di un repository GIT con più branch:

- il tuo
- quello del collega
- dev (o integrazione delle modifiche)
- produzione

Workflow

Esempio di workflow collaborativo:



Altri tip per ottimizzare il workflow:
`https://get-git.readthedocs.io/it/latest/obiettivo_7.html`

Branching

Con operazione di branching, in tutti i VCS, si intende l'operazione tramite il quale l'utente "si stacca" dal principale ramo di sviluppo e continua a lavorare senza intaccare il ramo principale. In molti VCS viene vista come operazione dispendiosa di risorse e tempo, e raramente fa parte di workflow.
In Git invece il branching è diventata una parte vitale del workflow di lavoro di molti utenti grazie alla semplicità nel creare nuovi branch e spostarsi tra di essi.



Branching

crea un nuovo branch chiamato "feature_x" e passa al nuovo branch usando

```
$ git checkout -b feature_x
```

ritorna di nuovo su master

```
$ git switch master
```

e cancella il branch creato in precedenza

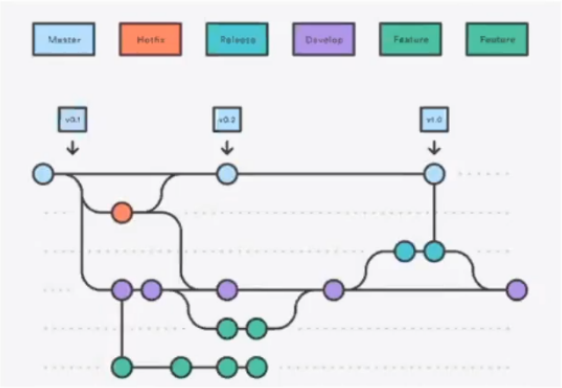
```
$ git branch -d feature_x
```

il branch non sarà disponibile agli altri fino a quando non verrà inviato al repository remoto

```
$ git push origin <branch>
```

Merge

Con operazione di merge, in tutti i VCS, si intende l'operazione opposta del branching tramite il quale si consolidano uno o più branch in un unico ramo di lavoro.



Remote

Per consentire agli utenti di lavorare in maniera distribuita e di collaborare su un repository Git abbiamo bisogno del cosidetto remote, ovvero di una versione del nostro repository Git gestita e in hosting su Internet o su un'altra rete. Le operazioni di sincronizzazione con il remote sono tre: fetch, pull e push

1. Una delle operazioni che si usano per sincronizzare la versione locale del repository con quella del remote è l'operazione di fetch, ovvero l'operazione di scaricare commit, file, e gli altri oggeti del database Git dal remote nella copia locale senza però modificare quest'ultima.
2. Un'altra operazione usata per sincronizzare la versione locale con il remote è quella di pull, ovvero l'operazione che oltre a scaricare inuovi dati dal remote come il fetch, li integra anche con la copia locale effettuando un merge automatico dove necessario e possibile (altrimenti verrà eseguita una merge request)
3. L'operazione di push invece è quell'operazione tramite il quale andiamo ad integrare le modifiche effettuate localmente nella versione del remote.

Remote

È bene notare che ogni operazione di sincronizzazione del repository, sia in entrata che in uscita, è legata ad un branch.
L'utente è libero di muoversi tra un branch e un altro nella working directory, ma non è possibile effettuare operazioni di sincronizzazione senza specificare quale branch del remote sincronizzare

Scaricare ed unire le modifiche

Per aggiornare il tuo repository locale alla commit più recente, esegui

```
$ git pull
```

Nella tua directory corrente per fare una fetch (recuperare) ed incorporare (merge) le modifiche fatte sul server remoto. Il comando pull cerca di auto-incorporare le modifiche locali con la copia nel server, ma questo non sempre è possibile.
Per incorporare un altro branch nel tuo branch attivo (ad esempio master), utilizza

```
$ git merge <branch>
```

Il comando di git "merge" combina le sequenze multiple di commit in un'unica storia unificata. Normalmente usato per due commit. L'algoritmo di GIT riesce ad effettuare senza problemi il merge di due rami e se solo se in solo uno dei due c'è una modifica alla nella stessa parte del file. In entrambi i casi git prova ad auto-incorporare le modifiche. Sfortunatamente, a volte questa procedura automatizzata non è possibile, ed in questo caso ci saranno dei conflitti.

Conflitti nel merge

Se due file hanno modficato la stessa parte del file il merge di git fallisce e viene modificato il file in questo modo:

```
here is some content not affected by the
   conflict
<<<<<<< master
this is conflicted text from master
=======
this is conflicted text from feature branch
```





Origin Master

>git merge



Effettuare il merge

Dato che sei tu il responsabile del merge Git mette in automatico i file nello stato "untracked". Una volta risolti i conflitti, si pososno mettere i suddetti file nello stato staged con:

```
$ git add <nomedelfile>
```

prima di immettere le modifiche, potrai anche visualizzarne un'anteprima eseguendo

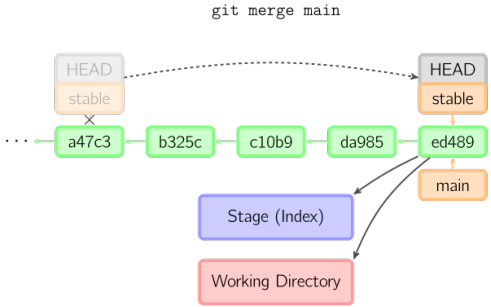
```
$ git diff <branch_sorgente> <branch_target>
```

Infine si effettua il commit:

```
$ git commit -m "Merge two branch"
```

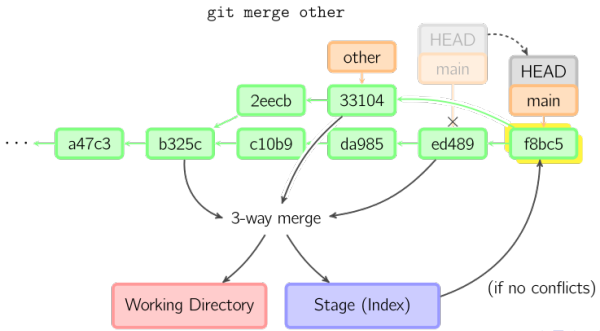
Effettuare il merge

Il caso banale è quello in cui l'altro commit è un antenato del commit corrente, non serve fare niente. Un altro caso semplice è quello in cui il commit corrente è un antenato dell'altro commit. Avremo quindi un merge fast-forward. La referenza viene semplicemente spostata e viene eseguito il checkout sul nuovo commit.



Effettuare il merge

Negli altri casi avremo un merge "reale". Si possono scegliere altre strategie ma quella di default è rappresentata dal merge "ricorsivo" che semplicemente prende il commit corrente (ed489 below), l'altro commit (33104), il loro antenato comune (b325c) ed esegue un merge a tre vie. Il risultato viene memorizzato nella working directory e nello stage, seguirà un commit con un genitore extra (33104).



Tags

È raccomandato creare dei tags nel caso in cui il software venga rilasciato. Questo è un concept già conosciuto, che esiste anche in SVN. Puoi creare un tag chiamato 1.0.0 eseguendo

```
$ git tag 1.0.0 1b2e1d63ff
```

la sequenza 1b2e1d63ff sta per i primi 10 caratteri del commit che si vuol referenziare tramite questo tag. Puoi ottenere l'id della commit tramite

```
$ git log
```

puoi anche utilizzare meno caratteri per l'id della commit, basta che sia unico.

1 Cos'è GIT
2 Architettura di GIT
3 GIT vs SVN
4 Installazione di GIT
5 Alcuni esempi
6 Workflow collaborativo
7 Fare i giocolieri con i commit
8 Utilizzo di GitHub, GitLab e Bitbucket
9 GIT e Jetbrains
10 Cosa non fa GIT
11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Differenze apportate ai commit

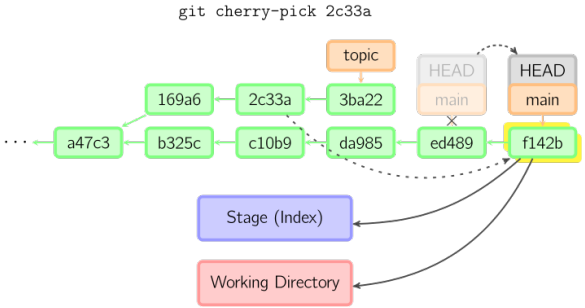
Per git, calcolare le modifiche apportate ai file da un commit all'altro non è poi difficile. Per esempio, puoi ricavarle con

```
git diff dev master
```

Con `git diff from to` chiedi a git «qual è l'elenco delle modifiche ai file che devo applicare a "from" perché il progetto diventi identico a quello fotografato in "to"»?

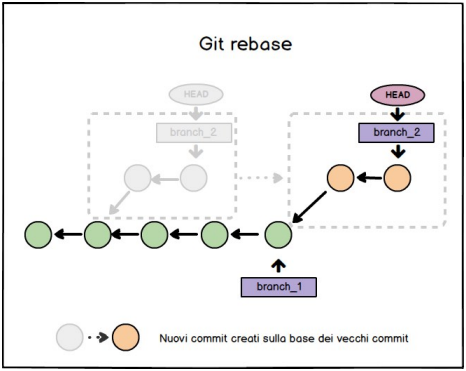
Cherry Pick

Con il comando cherry-pick puoi chiedere a git di calcolare le modifiche introdotte dal tuo commit e riapplicarle da qualche altra parte, per esempio, proprio su master, il nuovo commit ha lo stesso messaggio ma le modifiche sono applicate da un parent diverso.



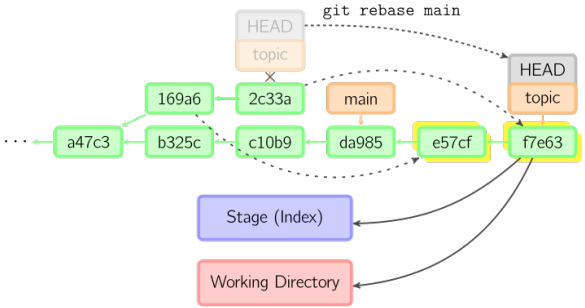
Rebase

Abbiamo visto come usare il comando git merge per fondere due branch. Git mette a disposizione un altro comando che si comporta in maniera simile, ovvero il comando git rebase che consente di integrare i commit di un branch nell'altro. Le funzionalità offerte da git rebase differiscono da git merge in alcuni aspetti chiave.
L'operazione di rebasing (rifondazione) consiste nello spostare una serie di commit in modo tale che il primo di questi sia preceduto da un diverso commit. In sostanza è un modo automatizzato per eseguire diversi cherry-pickin sequenza.



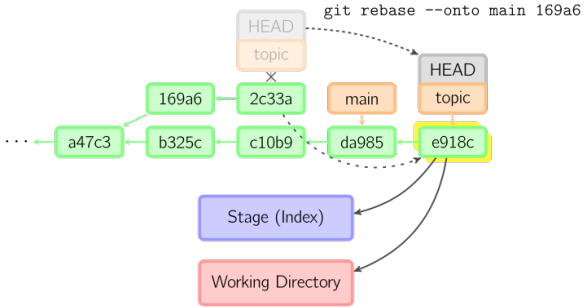
Rebase

N.B. in realtà i commit non vengono "spostati", in quanto ogni commit è un oggetto unico e immutabile identificato in maniera univoca da un valore di hash generato usando l'algoritmo SHA-1, ma viene creata un nuovo commit sulla base delle informazioni contenute nei vecchi oggetti (quindi rimangono le stesse informazioni, se non quella dei commit genitori e l'hash).



Rebase

Il comando precedente prende tutti i commit esistenti in topic ma non in main (precisamente 169a6 e 2c33a), li riporta sul main e poi sposta l'head del branch sulla nuova estremità.
Per limitare quanto andare all'indietro utilizzare l'opzione `--onto`. Il comando seguente riporta sul main i commit più recenti del branch corrente da 169a6 (escluso), precisamente 2c33a.



Rebase

Esiste anche `git rebase --interactive`, che permette di fare cose più complicate rispetto a riportare semplicemente commit, più precisamente eliminare, riordinare, modificare i commit.

Merge con squash

Il comando `git merge --squash` raggruppa tutte le commit in una sola. Al termine del merge è possibile cancellare il branch di lavoro senza alcun problema.
Supponiamo di avere un ramo bugfix e desideriamo applicare tutti questi commit al master:

```
git checkout master
git merge --squash bugfix
git commit
```

Questo vi porterà tutti i commit dal ramo bugfix, inglobandoli in 1 commit, e si fondono con il vostro ramo master.
Questo comando insieme al comando cherry-pick è particolarmente utile quando si hanno più release attualmente supportate (una LTS ed una "liscia") e dobbiamo realizzare bug-fix per entrambe le release.

Git & DevOps

Attraverso un potente modello di branching e vari workflow di collaborazione, negli anni l'uso e l'adozione di Git è cresciuta esponenzialmente rispetto ad altri VCS, arrivando ad essere il più usato e il più popolare soprattuto per come è possibile integrarlo come fondamenta di un processo DevOps.

1 Cos'è GIT
2 Architettura di GIT
3 GIT vs SVN
4 Installazione di GIT
5 Alcuni esempi
6 Workflow collaborativo
7 Fare i giocolieri con i commit
8 Utilizzo di GitHub, GitLab e Bitbucket
9 GIT e Jetbrains
10 Cosa non fa GIT
11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Utilizzo di GitHub, GitLab e Bitbucket

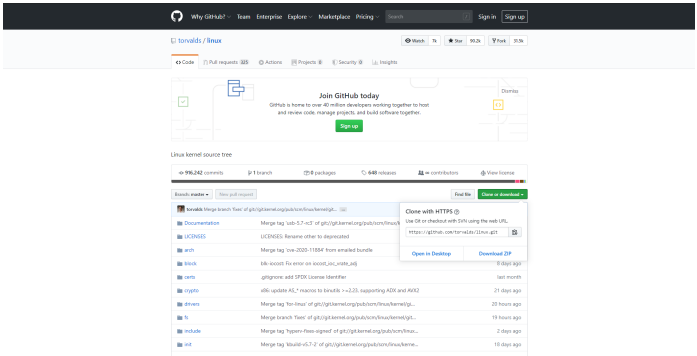
Questi sono alcuni servizi di hosting web-based per progetti che usano il sistema di controllo versione GIT. Il primo è comunemente noto per l'hosting di numerosi progetto open source, tra cui:

1. `https://github.com/torvalds/linux`
2. `https://github.com/microsoft/MS-DOS` (ovviamente GIT non c'era ai tempi dello sviluppo di MS-DOS e usavano i tasti CTRL+C e CTRL+V per fare lo storico delle versioni)

GitLab e Bitbucket sono invece orientati al settore business offrendo numerosi servizi extra on cloud ed apprezzati dalle imprese. Inoltre sia GitLab che Bitbucket permettono l'installazione in maniera on premise. Tutti e tre permettono la creazione di repository privati gratuiti.

Scarichiamo un repository da GitHub

Su GitHub troviamo il link (HTTPS o SSH) per scaricare il repository. Si noti che questo URL NON è l'URL della pagina web che si visita, ma l'url che si trova con il pulsante "Clone".




```
$ git clone https://gitlab.com/test/test.git
```

Utilizzo di GitLab

Creiamo un nuovo progetto su GitLab, dopo esserci registrati su GitLab (`https://gitlab.com/`), ricordo di usare una e-mail "decente" che possa identificate il programmatore e non furbissimo@gmail.com. Non stiamo scrivendo un software che hackera i server della NASA!
Premiamo il puslante "New Project" in alto a destra e creiamo il nostro progetto. Diamoli un nome, per esempio "test-GitLab" dopo averlo creato ci appaiono delle semplici "Command line instructions". Le eseguiamo per esercitarci su Git. Una volta finito creiamo un file dot gitgnore e "puliamo" con i comandi il repository da eventuali file non ammessi!
In maniera simile anche da GitLab un repo si scarica con git clone. Si noti che questo URL NON è l'URL della pagina web che si visita, ma l'url che si trova con il pulsante "Clone".

```
$ git clone https://github.com/torvalds/linux.git
```

Utilizzo di Bitbucket

Come GitLab e GitHub anche Bitbucket permette di le stesse azioni degli altri due

```
$ git clone https://bitbucket.org/chromiumembedded/cef.git
```

1 Cos'è GIT

2 Architettura di GIT

3 GIT vs SVN

4 Installazione di GIT

5 Alcuni esempi

6 Workflow collaborativo

7 Fare i giocolieri con i commit

8 Utilizzo di GitHub, GitLab e Bitbucket

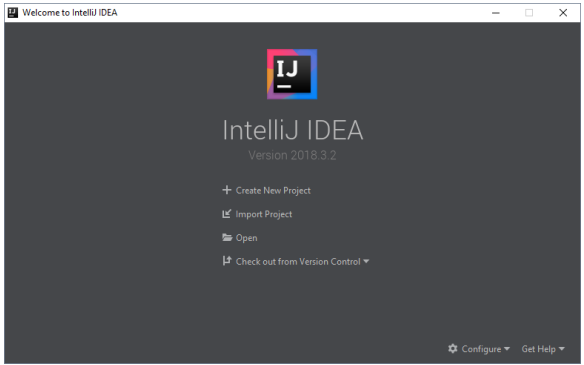
9 GIT e Jetbrains

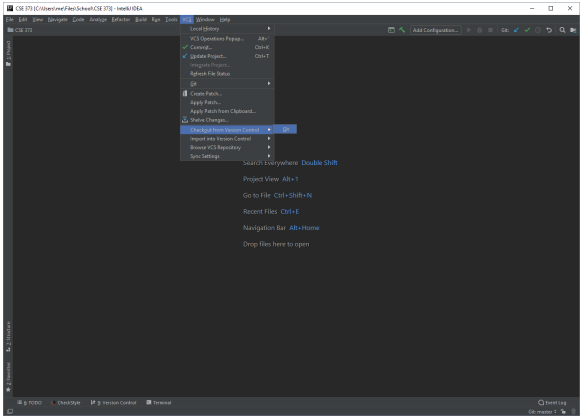
10 Cosa non fa GIT

11 Collaborazione attraverso GitHub, GitLab e Bitbucket

GIT e Jetbrains - Apertura di una repo

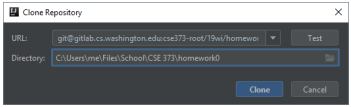
Una volta copiato il link al repo git. Se IntelliJ è aperto nella schermata di benvenuto, è sufficiente cliccare sull'opzione "Check out from Version Control". Altrimenti, se un progetto è già aperto, potete cliccare su "VCS" > "Checkout from Version Control" > "Git".



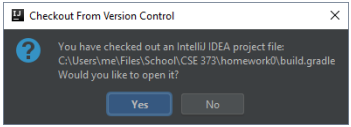


Git e Jetbrains - Apertura di una repo

In seguito, si aprirà una finestra in cui si dovrebbe inserire l'URL del repository che abbiamo precedente copiato (per esempio di GitLab, GitHub, ...) e una directory in cui salvare il repo clonato.



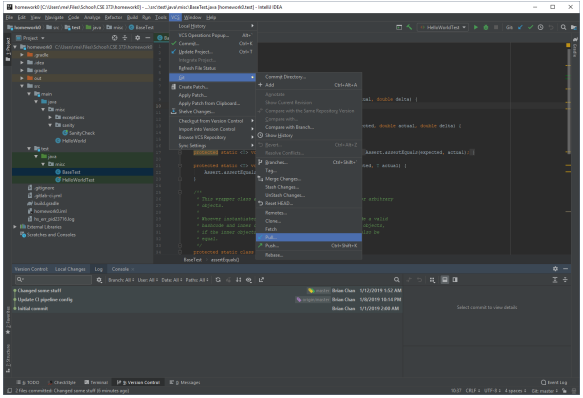
Se hai clonato correttamente uno dei nostri repository dei compiti a casa, IntelliJ dovrebbe apparire con un'altra finestra che ti chiede se vuoi aprire il progetto che hai appena clonato.

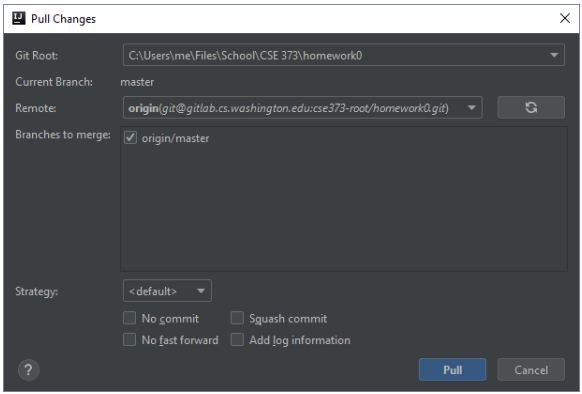


In linea generale bisogna premere Yes, ma ci sono alcuni casi in cui è meglio premere No.

Git e Jetbrains - Scaricare le modifiche

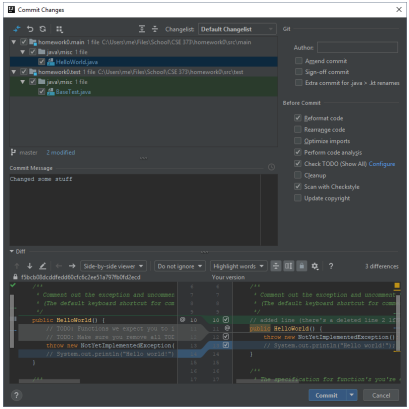
Prima di effettuare modifiche ad un repo con del codice è bene scaricare le modifiche effettuare nel frattempo per ridurre al minimo le possibilità di fusione dei conflitti. La funzionalità di pull in IntelliJ è in qualche modo nascosta nei menu e/o nel tasto destro. Clicca "VCS" > "Git" > "Pull..." nella finestra che si apre premere "Pull".





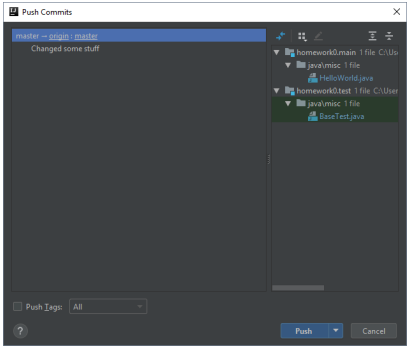
Git e Jetbrains - Committare le modifiche

Una volta di finito di fare le modifiche aprire il menu per il commit dal VCS o dal tasto destro. Apparirà una finestra. Selezionare quindi i file da committare e aggiungere un messaggio per il commit



Git e Jetbrains - Caricare le modifiche

Nella finestra precedente possiamo dire di caricare immediatamente le modifiche dal pulsante accanto a Commit, ma non sempre è la cosa ideale (tipo se siamo offline) o in altre situazioni. Quindi apriamo la finestra "Push" accessibile dal menu VCS o dal tasto destro.



Altre informazioni utili

Tutorial completo su come utilizzare Git su IntelliJ: `https://courses.cs.washington.edu/courses/cse373/19wi/resources/intellij/git/`
Tutorial completo su come utilizzare Git su Visual Studio: `https://docs.microsoft.com/en-us/azure/devops/repos/git/gitquickstart`

1 Cos'è GIT
2 Architettura di GIT
3 GIT vs SVN
4 Installazione di GIT
5 Alcuni esempi
6 Workflow collaborativo
7 Fare i giocolieri con i commit
8 Utilizzo di GitHub, GitLab e Bitbucket
9 GIT e Jetbrains
10 Cosa non fa GIT
11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Millenium Bug + 20

Si narra che negli anni 90 nel centro di calcolo UniPD, in via San Francesco 11, Padova. I tecnici abbiamo fixato il millenium bug shiftando di 20 anni le date (prassi assai comune) affinché il 1999 diventasse il 2019. Abbiano caricato le modifiche in produzione e salvato sul repository GIT con questo commit:

```
git commit -m "sort of fixed mbug, we'll be fine for another 20
   years"
git push
```

1. GIT non ti avvisa se hai fatto bug!
2. GIT non ha una AI che salva che fra 20 anni devi correggere un bug!
3. GIT è solamente un sistema di archiviazione del codice!

1 Cos'è GIT
2 Architettura di GIT
3 GIT vs SVN
4 Installazione di GIT
5 Alcuni esempi
6 Workflow collaborativo
7 Fare i giocolieri con i commit
8 Utilizzo di GitHub, GitLab e Bitbucket
9 GIT e Jetbrains
10 Cosa non fa GIT
11 Collaborazione attraverso GitHub, GitLab e Bitbucket

Collaborazione attraverso GitHub, GitLab e Bitbucket

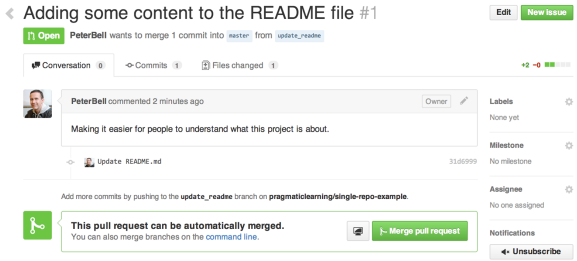
In queste slide inizieremo esaminando come collaborare direttamente sui repository GitHub, GitLab e Bitbucket. Dedicheremo un po' di tempo a approfondire la collaborazione utilizzando richieste pull, problemi e pagine GitHub.

Fork

Se volete contribuire direttamente a un progetto, dovete possederlo o esservi aggiunti come collaboratori. Se vuoi contribuire ad un progetto che non possiedi e su cui non sei un collaboratore, dovrai farne una copia su GitHub sotto il tuo account utente. Questo processo si chiama biforcazione. Una volta che hai biforcato un progetto, sarai in grado di fare tutte le modifiche che vuoi al tuo fork (copia) e potrai richiedere che le tue modifiche vengano incorporate nel progetto originale usando una richiesta di pull.
Anche se i fork sono un buon modo per accettare i contributi di persone con cui non si lavora regolarmente, sono un po' troppo ingombranti per l'uso quotidiano in un team che lavora a stretto contatto. Per questo motivo, probabilmente vorrete collaborare direttamente su un unico repository. Tuttavia, è comunque importante utilizzare le filiali e le richieste di pull per mantenere il vostro lavoro separato.

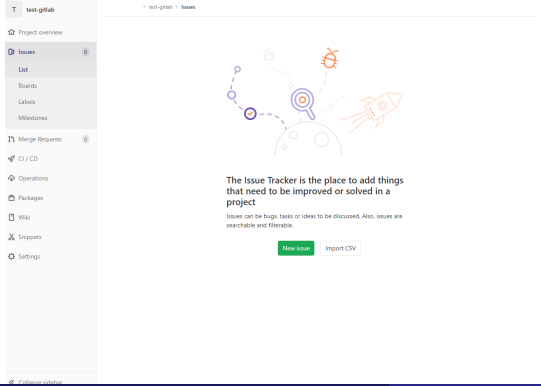
Pull request

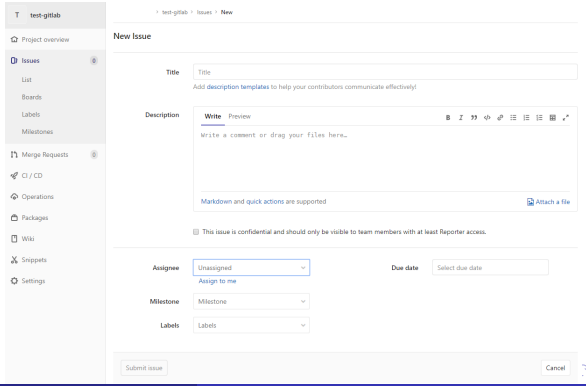
Spesso comunque nei repo si cerca di tenere il ramo master protetto, cioè modificabile solamente da chi gestisce il progetto e fare merge request dai rami di sviluppo. Per creare una pull request andrò nella pagina web del repo, farò clic sulla tab "Pull request" sul lato destro della pagina del progetto, quindi fare clic sul pulsante "Create pull request" per creare una Pull request. A sinistra c'è il ramo base e a destra il ramo dove sono state fatte le modifiche. Dopo aver selezionato un ramo fare clic sul pulsante "Create pull request" per creare una Pull request. Se non ci sono conflitti è possibile accettare la pull request senza problemi, altrimenti è necessario effettuare il merge.



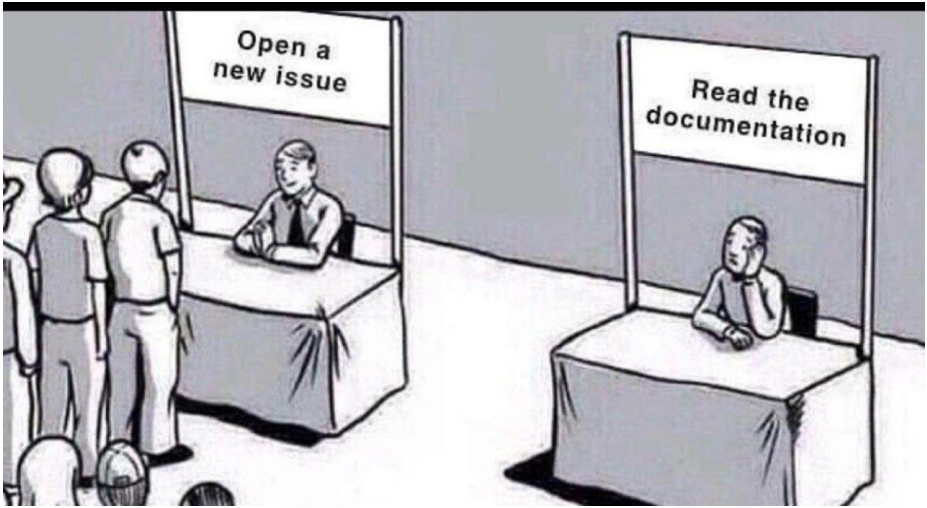
Issues

Un ultima funzionalità degna di nota di GitHub, GitLab ecc è la funzionalità Issues.
La funzionalità Issues permette la creazione di una baceca condivisa da tutti gli sviluppatori dove vengono inseriti i Bug e le funzionalità mancanti, con relativa descrizione, titolo e immagini. Permette di assegnare ad ogni sviluppatore dei determinati Issues da risolvere, e infine, segnali come risolti.





Come non usare gli Issues



Fonti

- https://rogerdudler.github.io/git-guide/index.it.html
- https://www.atlassian.com/git/tutorials/setting-up-a-repository
- https://git-scm.com/book/it/v1/Per-Iniziare-Basi-di-Git
- https://git-scm.com/book/it/v1/Per-Iniziare-Il-Controllo-di-Versione
- https://git-scm.com/book/it/v1/Per-Iniziare-Una-Breve-Storia-di-Git
- https://www.1and1.it/digitalguide/siti-web/programmazione-del-sito-web/git-vs-svn-controllo-di-versione-a-confronto/
- https://svnvsgit.com/
- https://medium.com/@prasoon2211/mercurial-vs-git-scaling-and-architecture-94f70bb206ef
- https://www.mercurial-scm.org/wiki/Revlog
- https://gvfs.io/